

Code Referee Addendum: Low-UI Convergence Failure

2026-02-24

Purpose

This addendum documents the likely convergence failure channels when unemployment insurance is set very low (especially $UI \rightarrow 0$), and lists the exact artifacts needed from the code owner to diagnose and fix the issue.

Status Update (2026-02-24)

- An isolated calibration workspace is now set up at: `projects/01_Necessity_Entrepreneurs/Calibration/A`
- Convergence guardrail patches are implemented in the AI copy of `cfv_red_final.cpp` (theta tolerance in aggregate stopping rule, runtime path/input checks, and denominator guards in fragile updates).
- A runnable compiler toolchain is now available in this environment (`C:/msys64/ucrt64/bin/g++.exe`).
- Full replication remains blocked by missing required solver inputs under `data/Input/CFV/`: `policy_functions_v17_in.txt`, `rnd_100k.txt`, `x_shk_iid.txt`, `input_pi_x_iid_1.txt`, `input_pi_x.txt`.

Likely Failure Channels (ranked)

1. **GE outer-loop instability at low UI.** With UI near zero, worker outside options collapse, occupational cutoffs move sharply, and fixed-point updates in wages/taxes/tightness can overshoot without damping.
2. **Infeasible-state spillovers.** Low UI increases probability mass near binding liquidity/consumption constraints; if feasibility guards are weak (negative/near-zero consumption), value updates can produce NaN/Inf or oscillation.
3. **Interpolation/indexing fragility.** Existing audit findings on education-index order and CEV indexing can amplify numerical noise and appear as non-convergence.
4. **Stopping-rule defect.** The previously flagged malformed aggregate convergence check can misclassify convergence/divergence and create false failures.
5. **Matching-object inconsistency.** If the implementation mixes objects corresponding to $p(\theta)$ and $q(\theta)$ inconsistently, effective hiring costs can become highly sensitive and destabilize updates when UI is low.

What Is Needed From Coauthor (minimum package)

1. **Code snapshot:** exact source file(s) used for runs plus commit hash/date.
2. **Two logs:** one convergent run (UI=0.4 benchmark) and one failing run (UI=0.0), each with full iteration prints.
3. **Parameter blocks:** all primitives and numerical controls for both runs:
 - policy params: UI replacement rate b , tax settings, transfer rules,
 - frictions: matching efficiency, elasticity, search cost parameters,
 - numerics: tolerances, max iterations, damping factors, update rules,
 - grids/interp settings: asset grid bounds, state grid sizes, interpolation method.
4. **Failure signature:** indicate whether failure is
 - inner VFI non-convergence,
 - policy oscillation,
 - GE outer-loop divergence,
 - NaN/Inf,
 - or infeasible choice sets.
5. **Intermediate outputs:** sequence of key aggregates by iteration (e.g., w , τ , θ , entrepreneurship rate, firm size) to locate where divergence starts.

Requested Debug Protocol

1. Reproduce both benchmark and failing runs with identical seed and logging format.
2. Verify feasibility guards in utility/resource constraints before maximization.
3. Separate inner-loop and outer-loop tolerances; confirm which loop fails first.
4. Add damping or line search to aggregate updates if not already present.
5. Use homotopy in UI: solve UI=0.4, then 0.3, 0.2, 0.1, 0.05, 0.0, warm-starting each step.
6. Re-run after applying previously flagged audit fixes (indexing and stopping-rule fixes) and compare convergence traces.

Decision Rule for “Ready to Recalibrate”

Proceed to new policy runs only after:

- benchmark and low-UI runs both converge under the same stopping logic,
- no NaN/Inf or feasibility violations are present,
- convergence traces are monotone/stable under at least two seeds,
- and reported moments are reproducible across reruns.